NeuroScript

AI-Based Handwriting Analysis for Neurological Detection

Formal Specification in Z Notation

Software Requirements Engineering — Final Year Project

Group Members: Munazza Kanwal (4475-FOC/BSSE-F22-A)
Sajjal Khalil (4488-FOC/BSSE-F22-A)
Hajira Gul (4454-FOC/BSSE-F22-A)
Areesha Abbasi (4508-FOC/BSSE-F22-A)

Submitted To: Course Instructor
Department: Faculty of Computing
Date: May 2026

Specification tool: LaTeX with `oz.sty` Z-schema package

Contents

Introduction

NeuroScript is an AI-powered web-based system that performs non-invasive early screening for neurological conditions — specifically OCD and ADHD — by analysing handwriting samples submitted by clinicians, students, or researchers. The system accepts real-time stylus/finger input or digitised paper samples, extracts neuromotor features (stroke speed, spacing, letter formation), runs an LSTM-based classification model, and produces an explainable prediction via SHAP/LIME.

This document presents a complete **formal Z specification** covering:

- Basic types and free-type enumerations
- State-space schema
- Initial-state schema
- Operational schemas for all major functions
- Error-reporting schemas and robust combined operations

Basic Types and Free-Type Declarations

Basic Sets

The following application-independent sets are introduced as basic types:

$$[USER, SAMPLE\_ID, FEATURE\_VECTOR, EXPLANATION]$$

`USER` The set of all registered system users (clinicians, researchers, students).

`SAMPLE_ID` Unique identifiers for every handwriting sample uploaded.

`FEATURE_VECTOR` Extracted neuromotor feature vectors (stroke speed, spacing, etc.).

`EXPLANATION` SHAP/LIME explanation objects attached to a prediction.

Free-Type Enumerations

$$ROLE ::= Clinician \mid Researcher \mid Student \mid Admin$$

$$CONDITION ::= Normal \mid PotentialADHD \mid PotentialOCD \mid Inconclusive$$

$$SAMPLE\_STATUS ::= Pending \mid Processing \mid Analysed \mid Archived$$

$$REPORT ::= success \mid alreadyExists \mid notFound \mid invalidInput \mid unauthorised \mid limitExceeded \mid pr$$

Global Constants

$$MAX\_SAMPLES\_PER\_USER == 500$$
$$MIN\_STROKE\_POINTS == 50$$

State-Space Schema

The central state of NeuroScript tracks registered users, uploaded samples, extracted features, model predictions, and generated explanations.

NeuroScript

$$
\begin{aligned}
&\mathit{registered\_users} : \mathbb{P}\ \mathit{USER} \\
&\mathit{user\_roles} : \mathit{USER} \pfun \mathit{ROLE} \\
&\mathit{samples} : \mathit{SAMPLE\_ID} \pfun \mathit{USER} \\
&\mathit{sample\_status} : \mathit{SAMPLE\_ID} \pfun \mathit{SAMPLE\_STATUS} \\
&\mathit{features} : \mathit{SAMPLE\_ID} \pfun \mathit{FEATURE\_VECTOR} \\
&\mathit{predictions} : \mathit{SAMPLE\_ID} \pfun \mathit{CONDITION} \\
&\mathit{explanations} : \mathit{SAMPLE\_ID} \pfun \mathit{EXPLANATION} \\
&\mathit{sample\_count} : \mathit{USER} \to \mathbb{N} \\
\hline
&\operatorname{dom} \mathit{user\_roles} = \mathit{registered\_users} \\
&\operatorname{dom} \mathit{samples} \subseteq \operatorname{dom} \mathit{sample\_status} \\
&\operatorname{dom} \mathit{features} \subseteq \operatorname{dom} \mathit{samples} \\
&\operatorname{dom} \mathit{predictions} \subseteq \operatorname{dom} \mathit{features} \\
&\operatorname{dom} \mathit{explanations} \subseteq \operatorname{dom} \mathit{predictions} \\
&\forall\, u : \mathit{registered\_users} \cdot \mathit{sample\_count}\ u = \#\{s : \operatorname{dom} \mathit{samples} \mid \mathit{samples}\ s = u\} \\
&\forall\, u : \mathit{registered\_users} \cdot \mathit{sample\_count}\ u \leq \mathit{MAX\_SAMPLES\_PER\_USER}
\end{aligned}
$$

Invariant explanation:

- Every user with a role is a registered user and vice versa.
- A sample can only have a status if it belongs to the samples map.
- Features may only exist for known samples; predictions require features; explanations require predictions.
- `sample_count` is derived and must not exceed the per-user limit.

Initial-State Schema

When the system is first deployed, no users, samples, or analysis results exist.

InitNeuroScript

$$
\begin{aligned}
&\mathit{NeuroScript} \\
\hline
&\mathit{registered\_users} = \varnothing \\
&\mathit{user\_roles} = \varnothing \\
&\mathit{samples} = \varnothing \\
&\mathit{sample\_status} = \varnothing \\
&\mathit{features} = \varnothing \\
&\mathit{predictions} = \varnothing \\
&\mathit{explanations} = \varnothing \\
&\mathit{sample\_count} = (\lambda\, u : \mathit{USER} \cdot 0)
\end{aligned}
$$

Operational Schemas

User Registration

Normal (success) case:

$$
\begin{array}{|l}
\textit{RegisterUser} \\
\hline
\Delta \textit{NeuroScript} \\
\textit{new\_user?} : \textit{USER} \\
\textit{role?} : \textit{ROLE} \\
\textit{result!} : \textit{REPORT} \\
\hline
\textit{new\_user?} \notin \textit{registered\_users} \\
\textit{registered\_users}' = \textit{registered\_users} \cup \{\textit{new\_user?}\} \\
\textit{user\_roles}' = \textit{user\_roles} \oplus \{\textit{new\_user?} \mapsto \textit{role?}\} \\
\textit{sample\_count}' = \textit{sample\_count} \oplus \{\textit{new\_user?} \mapsto 0\} \\
\textit{samples}' = \textit{samples} \\
\textit{sample\_status}' = \textit{sample\_status} \\
\textit{features}' = \textit{features} \\
\textit{predictions}' = \textit{predictions} \\
\textit{explanations}' = \textit{explanations} \\
\textit{result!} = \textit{success} \\
\hline
\end{array}
$$

Error case — user already exists:

$$
\begin{array}{|l}
\textit{UserAlreadyExists} \\
\hline
\Xi \textit{NeuroScript} \\
\textit{new\_user?} : \textit{USER} \\
\textit{result!} : \textit{REPORT} \\
\hline
\textit{new\_user?} \in \textit{registered\_users} \\
\textit{result!} = \textit{alreadyExists} \\
\hline
\end{array}
$$

Robust registration:

$$\textit{RRegisterUser} \triangleq (\textit{RegisterUser}) \lor \textit{UserAlreadyExists}$$

User Login

$$
\begin{array}{|l}
\textit{LoginUser} \\
\hline
\Xi \textit{NeuroScript} \\
\textit{user?} : \textit{USER} \\
\textit{result!} : \textit{REPORT} \\
\hline
\textit{user?} \in \textit{registered\_users} \\
\textit{result!} = \textit{success} \\
\hline
\end{array}
$$

LoginFail

$\Xi NeuroScript$
$user? : USER$
$result! : REPORT$

$user? \notin registered\_users$
$result! = notFound$

$RLoginUser \triangleq LoginUser \lor LoginFail$

Upload Handwriting Sample

A registered user uploads a new handwriting sample; the system assigns it a `Pending` status.

UploadSample

$\Delta NeuroScript$
$user? : USER$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$user? \in registered\_users$
$sid? \notin \mathrm{dom}\ samples$
$sample\_count\ user? < MAX\_SAMPLES\_PER\_USER$
$samples' = samples \cup \{sid? \mapsto user?\}$
$sample\_status' = sample\_status \cup \{sid? \mapsto Pending\}$
$sample\_count' = sample\_count \oplus \{user? \mapsto sample\_count\ user? + 1\}$
$registered\_users' = registered\_users$
$user\_roles' = user\_roles$
$features' = features$
$predictions' = predictions$
$explanations' = explanations$
$result! = success$

SampleAlreadyExists

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \in \mathrm{dom}\ samples$
$result! = alreadyExists$

UploadLimitExceeded

$\Xi NeuroScript$
$user? : USER$
$result! : REPORT$

$user? \in registered\_users$
$sample\_count\ user? \geq MAX\_SAMPLES\_PER\_USER$
$result! = limitExceeded$

UploaderNotFound

$\Xi NeuroScript$
$user? : USER$
$result! : REPORT$

$user? \notin registered\_users$
$result! = notFound$

$RUploadSample \triangleq UploadSample \lor SampleAlreadyExists \lor UploadLimitExceeded \lor UploaderNotF$

Preprocess and Extract Features

The system extracts a feature vector from a pending sample and updates its status to *Processing*.

ExtractFeatures

$\Delta NeuroScript$
$sid? : SAMPLE\_ID$
$fv? : FEATURE\_VECTOR$
$result! : REPORT$

$sid? \in \text{dom } samples$
$sample\_status\ sid? = Pending$
$sid? \notin \text{dom } features$
$features' = features \cup \{sid? \mapsto fv?\}$
$sample\_status' = sample\_status \oplus \{sid? \mapsto Processing\}$
$registered\_users' = registered\_users$
$user\_roles' = user\_roles$
$samples' = samples$
$sample\_count' = sample\_count$
$predictions' = predictions$
$explanations' = explanations$
$result! = success$

SampleNotFound

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \notin \text{dom } samples$
$result! = notFound$

InvalidSampleInput

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \in \text{dom } samples$
$sample\_status\ sid? \neq Pending$
$result! = invalidInput$

$$RExtractFeatures \triangleq ExtractFeatures \lor SampleNotFound \lor InvalidSampleInput$$

Classify / Predict Neurological Condition

The LSTM model analyses extracted features and produces a condition prediction.

ClassifySample

$\Delta NeuroScript$
$sid? : SAMPLE\_ID$
$condition? : CONDITION$
$result! : REPORT$

$sid? \in \mathrm{dom}\, features$
$sid? \notin \mathrm{dom}\, predictions$
$sample\_status\ sid? = Processing$
$predictions' = predictions \cup \{sid? \mapsto condition?\}$
$sample\_status' = sample\_status \oplus \{sid? \mapsto Analysed\}$
$registered\_users' = registered\_users$
$user\_roles' = user\_roles$
$samples' = samples$
$sample\_count' = sample\_count$
$features' = features$
$explanations' = explanations$
$result! = success$

FeaturesNotFound

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \notin \mathrm{dom}\, features$
$result! = notFound$

ClassificationError

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \in \mathrm{dom}\, features$
$sample\_status\ sid? \neq Processing$
$result! = processingError$

$$RClassifySample \triangleq ClassifySample \lor FeaturesNotFound \lor ClassificationError$$

Generate Explanation (SHAP / LIME)

GenerateExplanation

$\Delta NeuroScript$
$sid? : SAMPLE\_ID$
$exp? : EXPLANATION$
$result! : REPORT$

$sid? \in \text{dom}\ predictions$
$sid? \notin \text{dom}\ explanations$
$explanations' = explanations \cup \{sid? \mapsto exp?\}$
$registered\_users' = registered\_users$
$user\_roles' = user\_roles$
$samples' = samples$
$sample\_count' = sample\_count$
$features' = features$
$predictions' = predictions$
$sample\_status' = sample\_status$
$result! = success$

PredictionNotFound

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \notin \text{dom}\ predictions$
$result! = notFound$

$RGenerateExplanation \triangleq GenerateExplanation \lor PredictionNotFound$

Retrieve Prediction Result

A read-only query: the state does not change.

GetPrediction

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$condition! : CONDITION$
$result! : REPORT$

$sid? \in \text{dom}\ predictions$
$condition! = predictions\ sid?$
$result! = success$

PredictionMissing

$\Xi NeuroScript$
$sid? : SAMPLE\_ID$
$result! : REPORT$

$sid? \notin \text{dom}\ predictions$
$result! = notFound$

$RGetPrediction \triangleq GetPrediction \lor PredictionMissing$

Search Samples by Condition

Returns all sample IDs predicted as a given condition owned by a given user.

SearchByCondition

$$\Xi NeuroScript$$
$$user? : USER$$
$$cond? : CONDITION$$
$$results! : \mathbb{P}\, SAMPLE\_ID$$
$$result! : REPORT$$

$$user? \in registered\_users$$
$$results! = \{s : \mathrm{dom}\, predictions \mid samples\ s = user? \land predictions\ s = cond?\}$$
$$result! = success$$

UserNotFound

$$\Xi NeuroScript$$
$$user? : USER$$
$$result! : REPORT$$

$$user? \notin registered\_users$$
$$result! = notFound$$

$$RSearchByCondition \triangleq SearchByCondition \lor UserNotFound$$

Archive Sample

ArchiveSample

$$\Delta NeuroScript$$
$$sid? : SAMPLE\_ID$$
$$user? : USER$$
$$result! : REPORT$$

$$sid? \in \mathrm{dom}\, samples$$
$$samples\ sid? = user?$$
$$sample\_status\ sid? = Analysed$$
$$sample\_status' = sample\_status \oplus \{sid? \mapsto Archived\}$$
$$registered\_users' = registered\_users$$
$$user\_roles' = user\_roles$$
$$samples' = samples$$
$$sample\_count' = sample\_count$$
$$features' = features$$
$$predictions' = predictions$$
$$explanations' = explanations$$
$$result! = success$$

ArchiveUnauthorised

$$
\begin{array}{l}
\Xi NeuroScript \\
sid? : SAMPLE\_ID \\
user? : USER \\
result! : REPORT \\
\hline
sid? \in \mathrm{dom}\ samples \\
samples\ sid? \neq user? \\
result! = unauthorised
\end{array}
$$

NotAnalysedYet

$$
\begin{array}{l}
\Xi NeuroScript \\
sid? : SAMPLE\_ID \\
result! : REPORT \\
\hline
sid? \in \mathrm{dom}\ samples \\
sample\_status\ sid? \neq Analysed \\
result! = invalidInput
\end{array}
$$

$$RArchiveSample \triangleq ArchiveSample \lor SampleNotFound \lor ArchiveUnauthorised \lor NotAnalysedYet$$

Delete User Account

An admin can delete a user. All their samples and associated data are removed.

DeleteUser

$$
\begin{array}{l}
\Delta NeuroScript \\
admin? : USER \\
target? : USER \\
result! : REPORT \\
\hline
admin? \in registered\_users \\
user\_roles\ admin? = Admin \\
target? \in registered\_users \\
target? \neq admin? \\
registered\_users' = registered\_users \setminus \{target?\} \\
user\_roles' = \{target?\} \lhd user\_roles \\
samples' = \{s : \mathrm{dom}\ samples \mid samples\ s = target?\} \lhd samples \\
sample\_status' = \{s : \mathrm{dom}\ samples \mid samples\ s = target?\} \lhd sample\_status \\
features' = \{s : \mathrm{dom}\ samples \mid samples\ s = target?\} \lhd features \\
predictions' = \{s : \mathrm{dom}\ samples \mid samples\ s = target?\} \lhd predictions \\
explanations' = \{s : \mathrm{dom}\ samples \mid samples\ s = target?\} \lhd explanations \\
sample\_count' = \{target?\} \lhd sample\_count \\
result! = success
\end{array}
$$

DeleteUnauthorised

$\Xi NeuroScript$
$admin? : USER$
$result! : REPORT$

$admin? \in registered\_users$
$user\_roles\ admin? \neq Admin$
$result! = unauthorised$

$$RDeleteUser \triangleq DeleteUser \lor DeleteUnauthorised \lor UserNotFound$$

Summary of Schemas

Schema	Type	Purpose
`NeuroScript`	State Space	Core system state and invariants
`InitNeuroScript`	Initial State	All sets empty at start-up
`RegisterUser`	Δ Op	Add a new user with a role
`LoginUser`	Ξ Op	Authenticate existing user
`UploadSample`	Δ Op	Upload handwriting sample
`ExtractFeatures`	Δ Op	Preprocess & extract feature vector
`ClassifySample`	Δ Op	LSTM-based neurological prediction
`GenerateExplanation`	Δ Op	Produce SHAP/LIME explanation
`GetPrediction`	Ξ Op	Query prediction result (read-only)
`SearchByCondition`	Ξ Op	Search samples by predicted condition
`ArchiveSample`	Δ Op	Archive a fully analysed sample
`DeleteUser`	Δ Op	Admin removes user and all their data

All major operations have corresponding **error schemas** and are combined into **robust operations** using the Z disjunction operator ($\lor$), ensuring every possible input scenario produces a defined output via the `REPORT` free type.